COMPUTER GRAPHICS

Practical Programs in C (Turbo C / graphics.h)

A collection of 12 practical programs covering line and circle drawing, area filling, 2D transformations, clipping, and Bézier curves.

Environment: Turbo C with BGI graphics library

BGI path used in all programs: C:\\TURBOC3\\BGI

Note: Each program is standalone. Compile and run one program at a time. If your BGI folder is installed elsewhere, change the path in initgraph().

1. DDA Line Drawing Algorithm

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>
#include <stdlib.h>

int main()
{
    int gd = DETECT, gm;
    int x1, y1, x2, y2, dx, dy, steps, i;
    float x, y, xinc, yinc;

    printf("Enter x1 y1 x2 y2: ");
    scanf("%d%d%d%d", &x1, &y1, &x2, &y2);

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    dx = x2 - x1;
    dy = y2 - y1;

    steps = (abs(dx) > abs(dy)) ? abs(dx) : abs(dy);

    if(steps == 0)
    {
        putpixel(x1, y1, WHITE);
    }
    else
    {
        xinc = (float)dx / steps;
        yinc = (float)dy / steps;

        x = x1;
        y = y1;

        for(i = 0; i <= steps; i++)
        {
            putpixel((int)(x + 0.5), (int)(y + 0.5), WHITE);
            x = x + xinc;
            y = y + yinc;
        }
    }

    getch();
    closegraph();
    return 0;
}
```

2. Bresenham Line Drawing Algorithm

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>
#include <stdlib.h>

int main()
{
    int gd = DETECT, gm;
    int x1, y1, x2, y2, x, y;
    int dx, dy, sx, sy, p, i;

    printf("Enter x1 y1 x2 y2: ");
    scanf("%d%d%d%d", &x1, &y1, &x2, &y2);

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    dx = abs(x2 - x1);
    dy = abs(y2 - y1);

    sx = (x2 >= x1) ? 1 : -1;
    sy = (y2 >= y1) ? 1 : -1;

    x = x1;
    y = y1;

    if(dx >= dy)
    {
        p = 2 * dy - dx;

        for(i = 0; i <= dx; i++)
        {
            putpixel(x, y, WHITE);
            x = x + sx;

            if(p >= 0)
            {
                p = p + 2 * (dy - dx);
                y = y + sy;
            }
            else
            {
                p = p + 2 * dy;
            }
        }
    }
    else
    {
        p = 2 * dx - dy;

        for(i = 0; i <= dy; i++)
        {
            putpixel(x, y, WHITE);
            y = y + sy;

            if(p >= 0)
            {
                p = p + 2 * (dx - dy);
                x = x + sx;
            }
            else
            {
                p = p + 2 * dx;
            }
        }
    }

    getch();
    closegraph();
    return 0;
}
```

3. Midpoint Circle Drawing Algorithm

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

int main()
{
    int gd = DETECT, gm;
    int xc, yc, r, x, y, p;

    printf("Enter center xc yc and radius: ");
    scanf("%d%d%d", &xc, &yc, &r);

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    x = 0;
    y = r;
    p = 1 - r;

    while(x <= y)
    {
        putpixel(xc + x, yc + y, WHITE);
        putpixel(xc - x, yc + y, WHITE);
        putpixel(xc + x, yc - y, WHITE);
        putpixel(xc - x, yc - y, WHITE);

        putpixel(xc + y, yc + x, WHITE);
        putpixel(xc - y, yc + x, WHITE);
        putpixel(xc + y, yc - x, WHITE);
        putpixel(xc - y, yc - x, WHITE);

        if(p < 0)
        {
            p = p + 2 * x + 3;
        }
        else
        {
            p = p + 2 * (x - y) + 5;
            y--;
        }

        x++;
    }

    getch();
    closegraph();
    return 0;
}
```

4. Boundary Fill – 4-Connected Approach

The fill starts from a seed point inside a closed shape. Boundary fill stops at the boundary colour; flood fill replaces pixels matching the old colour.

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

void boundaryFill4(int x, int y, int boundary, int fill)
{
    int current = getpixel(x, y);

    if(current != boundary && current != fill)
    {
        putpixel(x, y, fill);

        boundaryFill4(x + 1, y, boundary, fill);
        boundaryFill4(x - 1, y, boundary, fill);
        boundaryFill4(x, y + 1, boundary, fill);
        boundaryFill4(x, y - 1, boundary, fill);
    }
}

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    setcolor(WHITE);
    rectangle(100, 100, 250, 250);

    boundaryFill4(150, 150, WHITE, RED);

    getch();
    closegraph();
    return 0;
}
```

5. Boundary Fill – 8-Connected Approach

The fill starts from a seed point inside a closed shape. Boundary fill stops at the boundary colour; flood fill replaces pixels matching the old colour.

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

void boundaryFill8(int x, int y, int boundary, int fill)
{
    int current = getpixel(x, y);

    if(current != boundary && current != fill)
    {
        putpixel(x, y, fill);

        boundaryFill8(x + 1, y, boundary, fill);
        boundaryFill8(x - 1, y, boundary, fill);
        boundaryFill8(x, y + 1, boundary, fill);
        boundaryFill8(x, y - 1, boundary, fill);

        boundaryFill8(x + 1, y + 1, boundary, fill);
        boundaryFill8(x - 1, y - 1, boundary, fill);
        boundaryFill8(x + 1, y - 1, boundary, fill);
        boundaryFill8(x - 1, y + 1, boundary, fill);
    }
}

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    setcolor(WHITE);
    rectangle(100, 100, 250, 250);

    boundaryFill8(150, 150, WHITE, RED);

    getch();
    closegraph();
    return 0;
}
```

6. Flood Fill – 4-Connected Approach

The fill starts from a seed point inside a closed shape. Boundary fill stops at the boundary colour; flood fill replaces pixels matching the old colour.

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

void floodFill4(int x, int y, int oldColor, int newColor)
{
    if(getpixel(x, y) == oldColor)
    {
        putpixel(x, y, newColor);

        floodFill4(x + 1, y, oldColor, newColor);
        floodFill4(x - 1, y, oldColor, newColor);
        floodFill4(x, y + 1, oldColor, newColor);
        floodFill4(x, y - 1, oldColor, newColor);
    }
}

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    setcolor(WHITE);
    rectangle(100, 100, 250, 250);

    floodFill4(150, 150, BLACK, RED);

    getch();
    closegraph();
    return 0;
}
```

7. Flood Fill – 8-Connected Approach

The fill starts from a seed point inside a closed shape. Boundary fill stops at the boundary colour; flood fill replaces pixels matching the old colour.

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

void floodFill8(int x, int y, int oldColor, int newColor)
{
    if(getpixel(x, y) == oldColor)
    {
        putpixel(x, y, newColor);

        floodFill8(x + 1, y, oldColor, newColor);
        floodFill8(x - 1, y, oldColor, newColor);
        floodFill8(x, y + 1, oldColor, newColor);
        floodFill8(x, y - 1, oldColor, newColor);

        floodFill8(x + 1, y + 1, oldColor, newColor);
        floodFill8(x - 1, y - 1, oldColor, newColor);
        floodFill8(x + 1, y - 1, oldColor, newColor);
        floodFill8(x - 1, y + 1, oldColor, newColor);
    }
}

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    setcolor(WHITE);
    rectangle(100, 100, 250, 250);

    floodFill8(150, 150, BLACK, RED);

    getch();
    closegraph();
    return 0;
}
```

8. Basic 2D Transformations – Translation, Scaling and Rotation

The original square is drawn in white and the transformed square in red. The square is defined around the mathematical origin, displayed at screen position (320,240).

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>
#include <math.h>

int main()
{
    int gd = DETECT, gm;
    int i, choice;
    float x[4] = {-50, 50, 50, -50};
    float y[4] = {-50, -50, 50, 50};
    float tx, ty, sx, sy, angle, rad, X, Y;

    printf("1. Translation\n");
    printf("2. Scaling\n");
    printf("3. Rotation\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if(choice == 1)
    {
        printf("Enter tx and ty: ");
        scanf("%f%f", &tx, &ty);
    }
    else if(choice == 2)
    {
        printf("Enter sx and sy: ");
        scanf("%f%f", &sx, &sy);
    }
    else if(choice == 3)
    {
        printf("Enter angle in degrees: ");
        scanf("%f", &angle);
        rad = angle * 3.14159 / 180.0;
    }
    else
    {
        printf("Invalid choice");
        getch();
        return 0;
    }

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    /* Draw original square in WHITE */
    setcolor(WHITE);
    for(i = 0; i < 4; i++)
        line(320 + x[i], 240 - y[i],
             320 + x[(i + 1) % 4], 240 - y[(i + 1) % 4]);

    /* Apply selected transformation */
    for(i = 0; i < 4; i++)
    {
        if(choice == 1)
        {
            x[i] = x[i] + tx;
            y[i] = y[i] + ty;
        }
        else if(choice == 2)
        {
            x[i] = x[i] * sx;
            y[i] = y[i] * sy;
        }
        else if(choice == 3)
        {
            X = x[i] * cos(rad) - y[i] * sin(rad);
            Y = x[i] * sin(rad) + y[i] * cos(rad);
            x[i] = X;
            y[i] = Y;
        }
    }

    /* Draw transformed square in RED */
    setcolor(RED);
    for(i = 0; i < 4; i++)
        line(320 + x[i], 240 - y[i],
             320 + x[(i + 1) % 4], 240 - y[(i + 1) % 4]);

    getch();
    closegraph();
    return 0;
}
```

9. 2D Transformations – Shearing and Reflection

The original square is drawn in white and the transformed square in red. The square is defined around the mathematical origin, displayed at screen position (320,240).

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

int main()
{
    int gd = DETECT, gm;
    int i, choice;
    float x[4] = {-50, 50, 50, -50};
    float y[4] = {-50, -50, 50, 50};
    float X, Y, shx, shy;

    printf("1. Reflection about X-axis\n");
    printf("2. Reflection about Y-axis\n");
    printf("3. Reflection about origin\n");
    printf("4. Shearing\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if(choice == 4)
    {
        printf("Enter shx and shy: ");
        scanf("%f%f", &shx, &shy);
    }
    else if(choice < 1 || choice > 3)
    {
        printf("Invalid choice");
        getch();
        return 0;
    }

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    /* Draw original square */
    setcolor(WHITE);
    for(i = 0; i < 4; i++)
        line(320 + x[i], 240 - y[i],
             320 + x[(i + 1) % 4], 240 - y[(i + 1) % 4]);

    /* Apply transformation */
    for(i = 0; i < 4; i++)
    {
        if(choice == 1)          /* Reflection about X-axis */
        {
            y[i] = -y[i];
        }
        else if(choice == 2)     /* Reflection about Y-axis */
        {
            x[i] = -x[i];
        }
        else if(choice == 3)     /* Reflection about origin */
        {
            x[i] = -x[i];
            y[i] = -y[i];
        }
        else if(choice == 4)     /* Shearing */
        {
            X = x[i] + shx * y[i];
            Y = y[i] + shy * x[i];
            x[i] = X;
            y[i] = Y;
        }
    }

    /* Draw transformed square */
    setcolor(RED);
    for(i = 0; i < 4; i++)
        line(320 + x[i], 240 - y[i],
             320 + x[(i + 1) % 4], 240 - y[(i + 1) % 4]);

    getch();
    closegraph();
    return 0;
}
```

10. Line Clipping – Cohen-Sutherland Algorithm

Uses a rectangular clipping window from (100,100) to (300,250). The original line is shown in yellow and the clipped line in red.

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

#define LEFT   1
#define RIGHT  2
#define TOP    4
#define BOTTOM 8

int xmin = 100, ymin = 100, xmax = 300, ymax = 250;

int getCode(int x, int y)
{
    int code = 0;

    if(x < xmin) code |= LEFT;
    if(x > xmax) code |= RIGHT;
    if(y < ymin) code |= TOP;
    if(y > ymax) code |= BOTTOM;

    return code;
}

int main()
{
    int gd = DETECT, gm;
    int x1, y1, x2, y2, c1, c2, c, x, y;
    int accept = 0;

    printf("Enter line endpoints x1 y1 x2 y2: ");
    scanf("%d%d%d%d", &x1, &y1, &x2, &y2);

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    /* Draw clipping window */
    setcolor(WHITE);
    rectangle(xmin, ymin, xmax, ymax);

    /* Draw original line for reference */
    setcolor(YELLOW);
    line(x1, y1, x2, y2);

    while(1)
    {
        c1 = getCode(x1, y1);
        c2 = getCode(x2, y2);

        if(c1 == 0 && c2 == 0)
        {
            accept = 1;
            break;
        }
        else if(c1 & c2)
        {
            break;
        }
        else
        {
            c = (c1 != 0) ? c1 : c2;

            if(c & TOP)
            {
                x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
                y = ymin;
            }
            else if(c & BOTTOM)
            {
                x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
                y = ymax;
            }
            else if(c & RIGHT)
            {
                y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
                x = xmax;
            }
            else
            {
                y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
                x = xmin;
            }

            if(c == c1)
            {
```

```
                x1 = x;
                y1 = y;
            }
            else
            {
                x2 = x;
                y2 = y;
            }
        }
    }

    if(accept)
    {
        setcolor(RED);
        line(x1, y1, x2, y2);
    }

    getch();
    closegraph();
    return 0;
}
```

11. Polygon Clipping – Sutherland-Hodgman Algorithm

Uses a rectangular clipping window from (100,100) to (300,250). Enter polygon vertices in order around the polygon.

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

#define MAX 50

int xmin = 100, ymin = 100, xmax = 300, ymax = 250;
int x[MAX], y[MAX], nx[MAX], ny[MAX];
int n;

int inside(int px, int py, int edge)
{
    if(edge == 0) return px >= xmin;  /* Left */
    if(edge == 1) return px <= xmax;  /* Right */
    if(edge == 2) return py >= ymin;  /* Top */
    return py <= ymax;                /* Bottom */
}

void intersect(int x1, int y1, int x2, int y2,
               int edge, int *ix, int *iy)
{
    float m;

    if(x2 != x1)
        m = (float)(y2 - y1) / (x2 - x1);
    else
        m = 0;

    if(edge == 0)  /* Left */
    {
        *ix = xmin;
        *iy = y1 + (int)(m * (xmin - x1));
    }
    else if(edge == 1) /* Right */
    {
        *ix = xmax;
        *iy = y1 + (int)(m * (xmax - x1));
    }
    else if(edge == 2) /* Top */
    {
        *iy = ymin;
        if(x2 != x1)
            *ix = x1 + (int)((ymin - y1) / m);
        else
            *ix = x1;
    }
    else /* Bottom */
    {
        *iy = ymax;
        if(x2 != x1)
            *ix = x1 + (int)((ymax - y1) / m);
        else
            *ix = x1;
    }
}

void clipPolygon(int edge)
{
    int i, j, count = 0;
    int x1, y1, x2, y2, ix, iy;
    int in1, in2;

    for(i = 0; i < n; i++)
    {
        j = (i + 1) % n;

        x1 = x[i];
        y1 = y[i];
        x2 = x[j];
        y2 = y[j];

        in1 = inside(x1, y1, edge);
        in2 = inside(x2, y2, edge);

        if(in1 && in2)
        {
            nx[count] = x2;
            ny[count++] = y2;
        }
        else if(in1 && !in2)
        {
            intersect(x1, y1, x2, y2, edge, &ix, &iy);
            nx[count] = ix;
            ny[count++] = iy;
        }
```

```
        else if(!in1 && in2)
        {
            intersect(x1, y1, x2, y2, edge, &ix, &iy);
            nx[count] = ix;
            ny[count++] = iy;

            nx[count] = x2;
            ny[count++] = y2;
        }
        /* If both are outside, add nothing. */
    }

    n = count;

    for(i = 0; i < n; i++)
    {
        x[i] = nx[i];
        y[i] = ny[i];
    }
}

int main()
{
    int gd = DETECT, gm;
    int i;

    printf("Enter number of polygon vertices (3 to 20): ");
    scanf("%d", &n);

    if(n < 3 || n > 20)
    {
        printf("Invalid number of vertices");
        getch();
        return 0;
    }

    printf("Enter vertices (x y):\n");
    for(i = 0; i < n; i++)
        scanf("%d%d", &x[i], &y[i]);

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    setcolor(WHITE);
    rectangle(xmin, ymin, xmax, ymax);

    setcolor(YELLOW);
    for(i = 0; i < n; i++)
        line(x[i], y[i], x[(i + 1) % n], y[(i + 1) % n]);

    clipPolygon(0); /* Left */
    clipPolygon(1); /* Right */
    clipPolygon(2); /* Top */
    clipPolygon(3); /* Bottom */

    if(n >= 3)
    {
        setcolor(RED);
        for(i = 0; i < n; i++)
            line(x[i], y[i], x[(i + 1) % n], y[(i + 1) % n]);
    }

    getch();
    closegraph();
    return 0;
}
```

12. Bézier Curve for n Control Points

Uses the Bernstein polynomial for any number of control points, up to MAX = 20. This program requires math.h for pow().

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>

#define MAX 20

long factorial(int n)
{
    long f = 1;
    int i;

    for(i = 1; i <= n; i++)
        f = f * i;

    return f;
}

long combination(int n, int r)
{
    return factorial(n) / (factorial(r) * factorial(n - r));
}

int main()
{
    int gd = DETECT, gm;
    int x[MAX], y[MAX];
    int n, i, k;
    float t, u, px, py;
    long c;

    printf("Enter number of control points (2 to %d): ", MAX);
    scanf("%d", &n);

    if(n < 2 || n > MAX)
    {
        printf("Invalid number of control points");
        getch();
        return 0;
    }

    printf("Enter control points (x y):\n");
    for(i = 0; i < n; i++)
        scanf("%d%d", &x[i], &y[i]);

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    /* Draw control polygon */
    setcolor(YELLOW);
    for(i = 0; i < n - 1; i++)
        line(x[i], y[i], x[i + 1], y[i + 1]);

    /* Draw Bezier curve using Bernstein polynomial */
    setcolor(WHITE);

    for(t = 0.0; t <= 1.0; t += 0.001)
    {
        u = 1.0 - t;
        px = 0.0;
        py = 0.0;

        for(k = 0; k < n; k++)
        {
            c = combination(n - 1, k);

            px = px + c * pow(u, n - 1 - k) * pow(t, k) * x[k];
            py = py + c * pow(u, n - 1 - k) * pow(t, k) * y[k];
        }

        putpixel((int)(px + 0.5), (int)(py + 0.5), WHITE);
    }

    getch();
    closegraph();
    return 0;
}
```